

PLANO DIRETOR MESTRE

Benchmark Agnóstico para Avaliação de Arquiteturas de IA e Machine Learning

Governança • Requisitos • Testes • Auditorias • Reprodutibilidade • Protocolo de Submissão

Campo	Definição
Status	Documento diretor inicial
Versão	1.0
Data-base	31 de agosto de 2026
Escopo	Desenvolvimento completo do benchmark, infraestrutura experimental, governança, validação, auditoria e submissões.
Princípio central	O benchmark mede capacidades observáveis; não mede a presença de módulos, nomes ou conceitos internos de uma arquitetura específica.

REGRA DE ISOLAMENTO

Durante a concepção, implementação e validação do benchmark, nenhuma arquitetura candidata deve influenciar sua construção.

Controle do documento

Item	Regra
Finalidade	Guiar o desenvolvimento técnico, científico, experimental e operacional do benchmark.
Unidade de avaliação	Capacidade funcional observável sob condições controladas.
Arquitetura-alvo	Nenhuma. Toda arquitetura entra apenas como submissão posterior.
Regra de avanço	Nenhum gate avança sem critérios de saída objetivos e evidência reproduzível.
Rastreabilidade	Todo resultado deve apontar para versão, hashes, config, seed, run_id, hardware e artefatos brutos.
Mudanças	Mudanças após freeze geram nova versão; resultados históricos não são reescritos.

Estrutura do documento

- 1. Missão e escopo
- 2. Regra de isolamento arquitetural
- 3. Filosofia científica
- 4. Governança
- 5. Separação física dos projetos
- 6. Interface universal
- 7. Política de adapters
- 8. Classes de capacidade
- 9-19. Baterias funcionais
- 20. Baselines
- 21. Validação do próprio benchmark
- 22. Política anti-Tailor-Made
- 23. Sistema de scoring
- 24. Estatística mínima
- 25. Registry experimental
- 26. Sanitização
- 27. Auditoria adversarial
- 28. Auditoria externa
- 29. Controle de mudanças
- 30. Versionamento
- 31. Política de claims
- 32. Processo de submissão
- 33. Gates
- 34. Ordem prática
- 35. Critério de maturidade
- 36. Objetivo maior

1. Missão

Construir uma plataforma independente, reproduzível e arquiteturalmente neutra capaz de avaliar agentes, arquiteturas cognitivas, sistemas de reinforcement learning, machine learning adaptativo e futuras arquiteturas de IA segundo capacidades funcionais mensuráveis.

Pergunta central do benchmark:

O que esta arquitetura consegue aprender, generalizar, preservar, adaptar, transferir e executar, sob quais custos e limitações?

O benchmark não deve inferir mecanismos internos a partir do resultado. Ele mede comportamento e desempenho sob protocolos controlados.

2. Regra de isolamento arquitetural

Esta é a regra mais importante do projeto:

Durante o desenvolvimento do benchmark, a arquitetura candidata não existe.

- A equipe do benchmark não deve analisar o código de arquiteturas candidatas.
- Nenhuma tarefa pode ser criada porque determinada arquitetura possui uma funcionalidade específica.
- Nenhuma métrica pode depender de estruturas internas exclusivas.
- Nenhum parâmetro pode ser escolhido para favorecer uma arquitetura conhecida.
- Nenhuma modificação pode ser feita no conjunto final após observar resultados de uma submissão.
- Arquiteturas candidatas entram somente após o primeiro benchmark congelado.

3. Filosofia científica

3.1 Capability-first

Medir comportamento e capacidade, não implementação. Um teste deve criar uma condição funcional e observar se o agente a resolve.

3.2 Hipótese antes do resultado

Hipótese, métricas, baselines, seeds, orçamento, exclusões e análise estatística são registrados antes da execução.

3.3 Controles positivos e negativos

Todo teste deve provar que discrimina a capacidade pretendida antes de ser usado em submissões.

3.4 Reprodutibilidade

Todo resultado deve ser reproduzível a partir de versão, hashes, configuração, seed e run_id.

3.5 Não antropomorfização

Capacidades recebem nomes operacionais, não psicológicos, salvo quando houver definição computacional explícita.

3.6 Resultados negativos são resultados

Falhas, regressões e hipóteses rejeitadas devem ser preservadas.

3.7 Perfil multidimensional

O benchmark não deve esconder fraquezas em um score único nas primeiras versões.

4. Governança

Função	Responsabilidade	Restrição
Conselho científico	Construtos, metodologia, validade estatística, métricas e aprovação de mudanças científicas.	Não desenvolver adapters de candidatos.
Engenharia do benchmark	Ambientes, runtime, API, registry, execução, testes e infraestrutura.	Não otimizar tarefas para submissões.

Função	Responsabilidade	Restrição
Equipe de validação	Leakage, shortcuts, reward hacking, comportamento degenerado e consistência estatística.	Atuar como red team do benchmark.
Equipe de submissão	Adapters, manifests e integração de candidatos após o freeze.	Sem alterar benchmark.
Auditor externo	Reprodução independente, auditoria de logs, hashes e conclusões.	Sem participação na implementação principal.

5. Separação física dos projetos

```
ai-benchmark-core/
ai-benchmark-submissions/
ai-benchmark-sealed/
```

ai-benchmark-core: API, ambientes públicos de desenvolvimento, métricas, baselines, documentação e validadores.

ai-benchmark-submissions: adapters, manifests, configurações e hashes das arquiteturas submetidas.

ai-benchmark-sealed: seeds finais, tarefas finais, perturbações ocultas, combinações OOD e parâmetros adversariais. Acesso restrito.

6. Interface universal

O benchmark deve operar predominantemente em caixa-preta. O contrato conceitual mínimo deve ser simples, auditável e extensível.

```
class Agent:
    def reset(self, specification): ...
    def observe(self, observation): ...
    def act(self): ...
    def learn(self, transition): ...
    def save(self): ...
    def load(self): ...
```

Nem toda arquitetura precisa implementar aprendizagem online. Capacidades opcionais devem ser declaradas no manifest de submissão.

7. Regra dos adapters

O adapter é exclusivamente um conversor de protocolo.

Permitido	Proibido
Conversão de dtype	Extração de features
Organização de tensor	Cálculo de estado escondido
Conversão de unidade	Interpretação semântica
Serialização	Seleção de informação relevante
Mapeamento de enumerações equivalentes	Adição de histórico não fornecido ao agente
Conversão de nomenclatura	Ground truth, hints ou correção de erros do agente

Regra de auditoria: se removermos o adapter e entregarmos diretamente uma representação equivalente, a capacidade do agente não deve mudar.

8. Classes de capacidade

ID	Dimensão
C1	Aprendizagem
C2	Eficiência amostral
C3	Generalização
C4	Adaptação a mudança
C5	Dependência temporal / memória funcional
C6	Planejamento
C7	Continual learning
C8	Robustez
C9	Transferência multidomínio
C10	Calibração de incerteza

ID	Dimensão
C11	Eficiência computacional

9. Bateria C1 - Aprendizagem

Objetivo: determinar se o sistema consegue adquirir uma política útil por experiência, sem depender de comportamento pré-programado específico para a tarefa.

- Aprendizado de contingências
- Exploração
- Adaptação
- Convergência
- Estabilidade entre seeds

Métricas principais: retorno, curva de aprendizagem, AUC, passos até limiar e variância entre seeds.

10. Bateria C2 - Eficiência amostral

Pergunta: quantas experiências o sistema precisa para aprender?

```
return / environment_interaction
interactions_to_threshold
```

A eficiência amostral é especialmente relevante em aplicações em que experimentar no ambiente é caro, lento ou arriscado.

11. Bateria C3 - Generalização

Estruturar quatro níveis independentes:

```
TRAIN
ID-HELDOUT
OOD
STRUCTURAL - TRANSFER
```

Nível	Objetivo
TRAIN	Desenvolvimento e aprendizagem permitidos.
ID-HELDOUT	Instâncias inéditas da mesma família distributiva.
OOD	Distribuição alterada sem alterar arbitrariamente a regra de avaliação.
STRUCTURAL-TRANSFER	Mesma estrutura causal fundamental em representação, contexto ou combinação diferente.

Métricas: generalization gap, zero-shot transfer, few-shot adaptation e recuperação após adaptação.

12. Bateria C4 - Adaptação a mudanças

Durante uma execução, o ambiente pode modificar dinâmica, recompensa, sensores, disponibilidade de ações ou relações causais sem aviso explícito.

```
detection_latency
adaptation_latency
post_change_regret
recovery_rate
```

O teste deve distinguir adaptação funcional de simples persistência da política anterior.

13. Bateria C5 - Dependência temporal

Criar tarefas em que uma informação relevante apareça antes da decisão e deixe de estar diretamente observável quando a decisão for exigida.

```
t=2   informação X
...
t=50  decisão depende de X

Atrasos: 1, 5, 20, 50, 100, 500 passos
```

Adicionar interferência e informações irrelevantes. Medir retenção, precisão, resistência à interferência e reconstrução incorreta.

O benchmark não pergunta se o agente 'tem memória'. Ele cria problemas em que algum mecanismo funcional de retenção é necessário.

14. Bateria C6 - Planejamento

Criar decisões com consequências atrasadas e conflitos entre recompensa imediata e retorno futuro.

```
A -> +10 agora -> -50 depois
B -> -5 agora -> +80 depois
```

Variar o horizonte e alterar relações causais. Medir retorno, regret, horizonte efetivo e capacidade de replanejamento.

O benchmark jamais exige um módulo chamado world model. Apenas mede se o comportamento demonstra capacidade funcional equivalente.

15. Bateria C7 - Continual Learning

```
Aprendizagem: A -> B -> C -> D
Reteste:      A -> B -> C -> D
```

Avaliar retenção, forgetting, forward transfer, backward transfer e plasticidade. O protocolo deve permitir medir ganho em novas tarefas sem esconder destruição de competências antigas.

16. Bateria C8 - Robustez

- Ruído
- Observações ausentes
- Sensores enviesados
- Delay
- Outliers
- Falha parcial de ações
- Perturbações
- Adversarial distribution shift

Gerar a curva degradação x intensidade da perturbação. Robustez não significa desempenho perfeito; significa degradação mensurável, controlada e recuperável.

17. Bateria C9 - Transferência multidomínio

A mesma arquitetura congelada deve ser utilizada em famílias estruturalmente diferentes. Somente o adapter pode mudar.

```
Domain A - grid/discrete
Domain B - continuous dynamics
Domain C - resource management
Domain D - partial observability
Domain E - industrial process
```

Medir zero-shot, few-shot, velocidade de adaptação, desempenho final e transferência de conhecimento.

O core do agente não pode ser alterado entre domínios.

18. Bateria C10 - Incerteza

Quando a arquitetura fornecer confiança probabilística, medir Brier score, Expected Calibration Error (ECE), calibration curve e selective accuracy.

Quando não fornecer confiança explicitamente, registrar NOT_SUPPORTED em vez de FAIL. O benchmark não deve obrigar todas as arquiteturas a possuir o mesmo mecanismo interno.

19. Bateria C11 - Eficiência computacional

- CPU time
- GPU time
- RAM
- VRAM
- Energia, quando mensurável
- Número de parâmetros
- Tempo de treinamento
- Tempo de inferência
- Interações ambientais

Todo relatório deve separar desempenho bruto de desempenho por recurso. O benchmark não deve confundir maior capacidade computacional com melhor arquitetura.

20. Baselines

Conjunto mínimo de referência, aplicado conforme pertinência técnica:

Random
Rule-based
Tabular Q-learning
DQN
PPO
Recurrent agent
Model-based agent

Nem todos os baselines precisam participar de todos os testes. A regra é selecionar controles tecnicamente adequados ao construto.

21. Validação do próprio benchmark

Critério	Pergunta de validação
Sensibilidade	Detecta melhora real?
Especificidade	Evita premiar sistemas sem a capacidade?
Discriminação	Separa arquiteturas funcionalmente diferentes?
Confiabilidade	Repete conclusões sob repetição controlada?
Robustez	Pequenas variações de seed não alteram conclusões centrais?
Shortcuts	Existe solução trivial que contorna o construto?
Leakage	Há informação de teste acessível durante treino/desenvolvimento?

Se qualquer item crítico falhar, o status correto é TEST_INVALID. O agente não deve ser responsabilizado por um teste incapaz de medir o construto.

22. Política anti-Tailor-Made

1. Nenhum desenvolvedor de uma arquitetura candidata pode usar o conjunto final antes do freeze.
2. Ambientes finais são congelados.
3. Métricas e pesos são congelados.
4. Seeds finais são seladas.
5. Budgets são congelados.
6. Somente depois uma submissão pode ser adicionada.

Após observar a primeira submissão, nenhuma mudança no benchmark v1 é permitida. Correções materiais geram benchmark-v2; v1 permanece preservado.

23. Sistema de scoring

Não criar inicialmente um único 'AI Score'. O resultado científico deve ser um vetor de capacidades.

Learning	0.84
Generalization	0.71
Adaptation	0.92
Temporal dependency	0.77
Planning	0.65
Continual learning	0.82
Robustness	0.74
Transfer	0.68
Compute efficiency	0.90

Perfis de aplicação podem ponderar as dimensões depois, mas essa ponderação pertence à decisão de engenharia, não ao benchmark científico.

24. Estatística mínima

- Múltiplas seeds
- Intervalos de confiança
- Tamanho de efeito
- Comparações pareadas quando possível
- Distribuição completa
- Análise de outliers
- Preregistro

```
mean
median
std
CI95
effect_size
win_rate
```

Não publicar somente médias. Resultados críticos devem preservar distribuição, incerteza e variabilidade entre execuções.

25. Registry experimental

```
run_id
timestamp
benchmark_version
environment
scenario
seed
submission
model_hash
adapter_hash
config_hash
hardware
software
start_time
end_time
status
```

Estrutura mínima de artefatos:

```
raw/
derived/
logs/
metrics/
manifest/
```

Dados derivados nunca substituem dados brutos. Todo número publicado deve ser rastreável até a execução correspondente.

26. Política de sanitização

- Nenhum dado de arquitetura candidata dentro do benchmark.
- Nenhum comentário ou código contendo comportamento esperado de determinada submission.
- Nenhuma seed final usada durante desenvolvimento.
- Nenhum condicional por submission.
- Nenhum path exclusivo de candidato.
- Nenhum hidden hint.
- Nenhuma dependência implícita.

Deve existir um `benchmark_sanitizer` automatizado executado em cada release candidate.

27. Auditoria adversarial

Antes do release, uma equipe diferente deve tentar derrotar o benchmark.

- Obter score alto sem aprender.
- Explorar bugs e inconsistências de reset.
- Memorizar seeds ou detectar IDs de teste.
- Inferir a distribuição final por metadados.
- Abusar do adapter.
- Explorar a função de reward ou a métrica.
- Provocar leakage.
- Usar comportamento degenerado que maximize score sem satisfazer o construto.

Objetivo: tentar quebrar o teste, não o agente.

28. Auditoria externa

O release v1 só deve receber status `VALIDATED` quando um terceiro independente conseguir:

1. Baixar e reconstruir o benchmark.
2. Executar os baselines.
3. Reproduzir as estatísticas.
4. Verificar hashes e manifests.
5. Chegar às mesmas conclusões metodológicas.

Antes disso, o status deve permanecer `RESEARCH_CANDIDATE`.

29. Controle de mudanças

```
CHANGE-ID
motivação
impacto
testes afetados
compatibilidade
risco de viés
resultado esperado
```

Mudanças em reward, ambiente, distribuição, métrica, scoring ou adapter protocol exigem análise científica formal.

30. Versionamento

```
v1.0
v1.1
v1.2
v2.0
```

Tipo	Regra
Patch	Corrige bug sem alterar o resultado científico esperado.
Minor	Adiciona teste ou capacidade mantendo comparabilidade documentada.
Major	Muda construto, protocolo, distribuição ou validade comparativa.

Resultados de versões major diferentes não devem ser comparados diretamente sem estudo de equivalência.

31. Política de claims

Permitido	Proibido
A arquitetura A apresentou maior eficiência amostral que B nesta bateria.	A é mais inteligente.
A apresentou transferência significativa em três domínios avaliados.	A pensa melhor.
A foi mais robusta sob a perturbação X no protocolo Y.	A é consciente.
A superou B sob o budget e métricas pré-registrados.	A é uma IA superior em sentido geral.

Toda conclusão deve permanecer vinculada ao construto, versão e protocolo que a sustentam.

32. Processo de submissão

Etapa	Ação
S1 - Freeze	Congelar código e pesos.
S2 - Manifest	Declarar versão, hardware, treinamento, recursos, dependências e capabilities opcionais.
S3 - Adapter	Implementar exclusivamente a interface.
S4 - Audit adapter	Validar ausência de inteligência adicional ou leakage.
S5 - Development run	Executar somente conjunto público.
S6 - Submission freeze	Gerar hashes de todos os artefatos.
S7 - Sealed evaluation	Executar conjunto oculto.
S8 - Report	Publicar automaticamente resultados e incertezas.

33. Gates do desenvolvimento

Gate	Marco	Critério de saída
B0	Charter	Governança e regras aprovadas.
B1	Construct Definition	Capacidades operacionalmente definidas.
B2	Universal API	Contrato neutro funcionando.
B3	Experimental Infrastructure	Registry, rastreabilidade e reprodução funcionando.
B4	Baselines	Controles implementados e validados.
B5	Core Tasks	Learning e generalization.
B6	Temporal Tasks	Dependência temporal e planning.
B7	Adaptation	Drift e continual learning.
B8	Robustness	Perturbações e OOD.
B9	Multidomain	Transferência entre famílias de problemas.
B10	Efficiency	Resource accounting.
B11	Benchmark Validation	Construct validity e discriminação.
B12	Adversarial Audit	Red team concluído.
B13	Sealed Set	Avaliação final congelada.
B14	Release Candidate	Nenhuma arquitetura-alvo testada.
B15	Independent Replication	Terceiro reproduz o benchmark.
B16	Submission Phase	Somente agora candidatos entram.

34. Ordem prática de desenvolvimento

```

B0  Governança
|
B1  Definições científicas
|
B2  API universal
|
B3  Infraestrutura experimental
|
B4  Baselines
|
B5-B10 Baterias
|
B11 Validação das baterias
|
B12 Red team
|
B13 Testes selados
|
B14 Freeze
|
B15 Auditoria independente
|
===== BARREIRA =====
|
B16 Primeira submissão

```

Antes da barreira, nenhuma arquitetura candidata participa.

35. Critério final de maturidade

O benchmark só deve ser considerado arquiteturalmente agnóstico quando uma nova arquitetura puder ser submetida exigindo apenas respostas operacionais:

- Qual é o formato das observações?
- Qual é o formato das ações?
- Como o agente é inicializado e reiniciado?
- A aprendizagem é online, offline ou inexistente?

Com essas informações, cria-se um adapter e executa-se a avaliação. Se for necessário modificar ambientes, rewards ou métricas para acomodar uma arquitetura, o benchmark ainda não é suficientemente agnóstico.

36. Objetivo maior

O projeto deve evoluir para uma infraestrutura de engenharia capaz de comparar arquiteturas de IA segundo perfis funcionais e econômicos, e não apenas segundo um ranking genérico.

Pergunta final de engenharia: qual arquitetura apresenta o melhor perfil funcional, estatístico e computacional para esta aplicação específica?

A condição metodológica permanece invariável: o benchmark é desenvolvido, validado e congelado primeiro; a arquitetura avaliada chega depois.

Declaração final de controle

Este documento deve orientar o desenvolvimento do benchmark como um sistema científico independente. Qualquer decisão que introduza conhecimento específico de uma arquitetura candidata antes do freeze deve ser registrada como potencial contaminação metodológica e bloqueia o avanço do gate correspondente.